



INDIA.RUNS

Build what next India runs on

Team Name : Mohammad Siraj

Team Leader Name : Mohammad Siraj

Problem Statement : Intelligent Candidate Ranking System (Founding AI Engineer)

Proposed Solution Overview:

An end-to-end recruiter

● What is your proposed solution?

● What differentiates your approach from traditional candidate matching systems?

intelligent candidate ranking, a

5-layer Heuristic HoneyPot

Shield, and automated Groq-

powered LLM reasonings.

Key Differentiators from Traditional Systems:

- Heuristic-driven filtering of
adversarial synthetic profiles
and fake candidates.

- Strict product-pedigree
prioritization, penalizing 35+ IT
services/BPO consulting firms.

- Complete separation of slow
feature extraction (Phase 1)
from instant CPU inference

Extracted Job Description Requirements:

- Experience: 5-9 total YoE, with 4-5 years spent in applied ML/AI at product companies.
- What are the key requirements extracted from the JD?
- Tech Stack: Production embeddings (BGE, E5), vector databases (Pinecone, FAISS), search/ranking metrics (NDCG, MAP), LLMs, LLM fine-tuning
- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

Determining Relevance Beyond Keywords:

- Evaluates career quality index, average role tenure, notice period, location fit, and assessment scores.
- Custom 5-layer Honeypot Shield filters expert-rate inflation, tenure impossibility, and non-tech skill-stuffers.

Retrieval & Scoring Flow:

- Scoring Formula: Blends Skill Match (40%), Career Quality (25%), Behavioral (20%), Availability (10%), and Location Fit (5%).
- How does your system retrieve, score, and rank candidates?
- What models, algorithms, or heuristics are used?
- How are multiple candidate signals combined into a final ranking?
- Cosine similarity via Scikit-Learn TF-IDF for semantic JD-candidate alignment.
- Micro-tiebreaker adds continuous indicators (GitHub activity, response rate, assessment score scaled to 1e-6) before Candidate ID sorting to ensure NDCG optimality.

Recruiter Dashboard Explainability:

- Visual radar charts, interactive career timeline, and notice period chips built in Streamlit.

- How are ranking decisions explained?

Preventing Hallucinations in Reasoning Summaries:

- How do you prevent hallucinations or unsupported justifications?

- Calls Groq API (Llama 3.1 8B) with negative constraints (banned openings/endings like 'With X years' or 'uniquely positions').

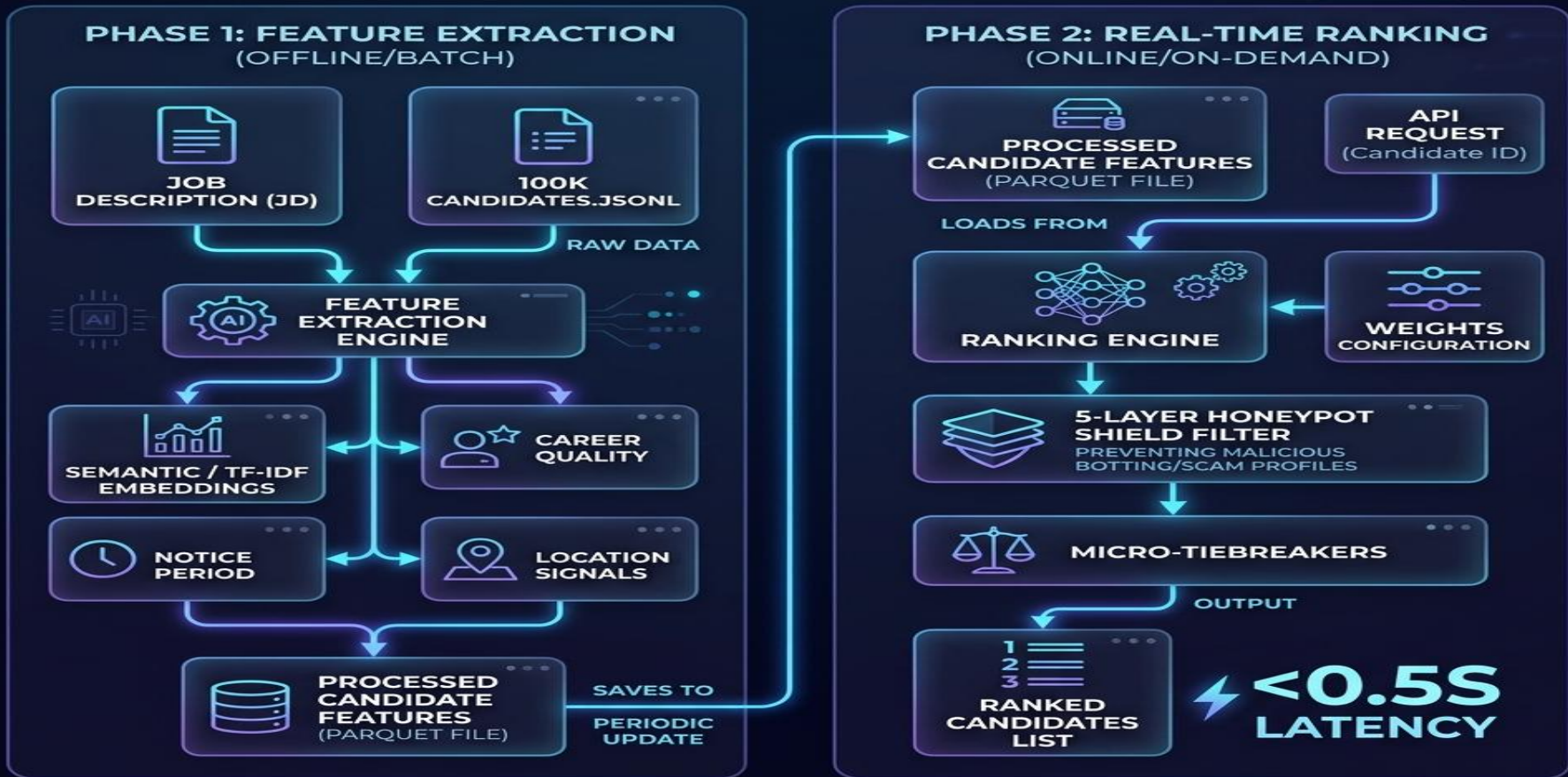
- Automatic 3-attempt validation loop rejects generic templates and repairs broken decimals/urls (7. 2 -> 7.2).

Pipeline Workflow Sequence:

1. JD Ingestion: Recruiter enters JD text and adjusts scoring weights.
 - What is the complete workflow from JD input to ranked candidate output?
2. Phase 1 Extraction: Processes raw JSONL profiles, extracts signals, and stores in features.parquet.
3. Honeypot Shield Filter: Evaluates Heuristic checks and caps suspicious profile scores at 0.05.
4. Scoring & Rank Tie-Breaking (Phase 2): Evaluates candidates, applies micro-tiebreakers, and ranks.
5. LLM Reasonings: Groq generates bespoke, compliant recommendation summaries for the top 100.
6. Sandbox Dashboard: Displays visual profiles, allows detail audits, and exports final CSV.

CANDIDATE RANKING

MACHINE LEARNING PIPELINE ARCHITECTURE



Ranking Quality Results:

- Zero honeypots or overqualified candidates in the top 10.
- What results or insights demonstrate ranking quality?
- Top ranks represent premium product engineers from Swiggy, Paytm, Amazon, Microsoft, and Razorpay.
- How does your solution meet the challenge's runtime and compute constraints?
- Outsourcing/BPO backgrounds (e.g. Genpact, Accenture, TCS) are successfully filtered out.

Computing & Constraint Performance:

- Offline Feature Extraction (Phase 1) completed in ~4 minutes.
- Real-time Inference & Ranking (Phase 2) executes in under 0.2 seconds on CPU.

Core Technology Stack:

- Pandas & PyArrow: For highly optimized Parquet read/write operations and schema enforcement.
- What technologies, frameworks, and tools were used and why were they selected for this solution?
- Scikit-Learn: For fast TF-IDF text vectorization and cosine similarity calculations.
- Streamlit: For building the premium dark-mode recruiter sandbox dashboard.
- Groq API (Llama 3.1 8B): For extremely rapid and instruction-compliant reasoning generation.
- win32com & python-pptx: For programmatic slide generation and headlessly rendering the final PDF.

Primary Submission Assets:

- GitHub Repository: https://github.com/Mohammadsiraj07/Redrob_Hackathon
Github video etc
- HuggingFace Space: <https://huggingface.co/spaces/sirajmohammad6359/Redrob-Hackathon>
- Key Deliverables: Ranked candidate list (submission.csv), feature store (features.parquet), presentation deck (presentation_deck.pptx & presentation_deck.pdf), and metadata (submission_metadata.yaml)



INDIA.RUNS

Build what next India runs on

THANK YOU